

Testbench in VHDL

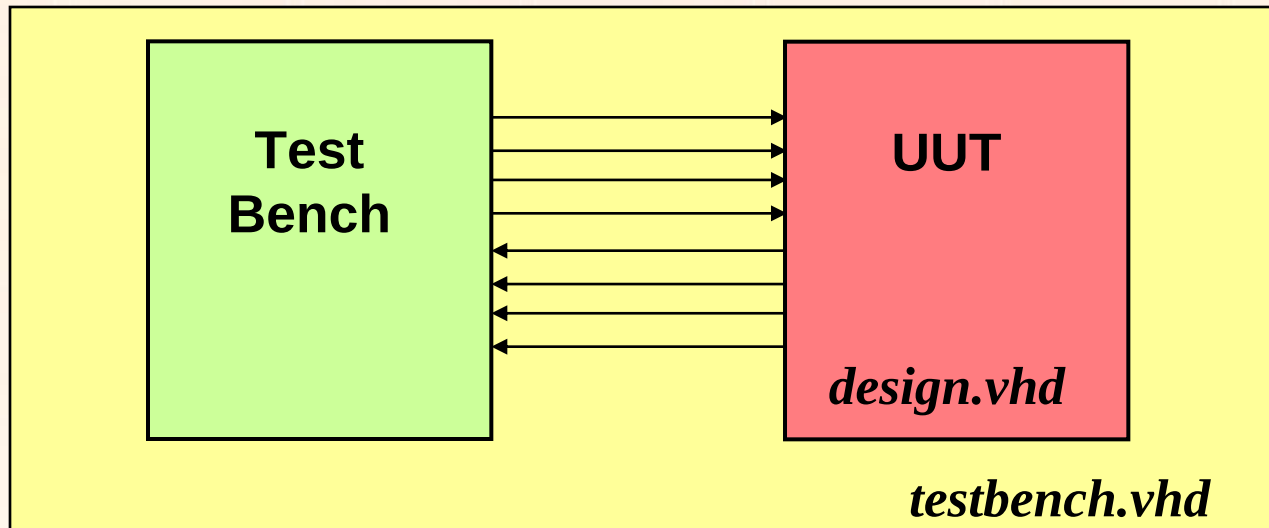
Lecture 6



Electronic and Communication Engineering Department
ECE 307 Introduction to Digital Systems Lab
Dr. Ammar Jallawi Mahmood
Ammar.karkar@uokufa.edu.iq

VHDL testbenches

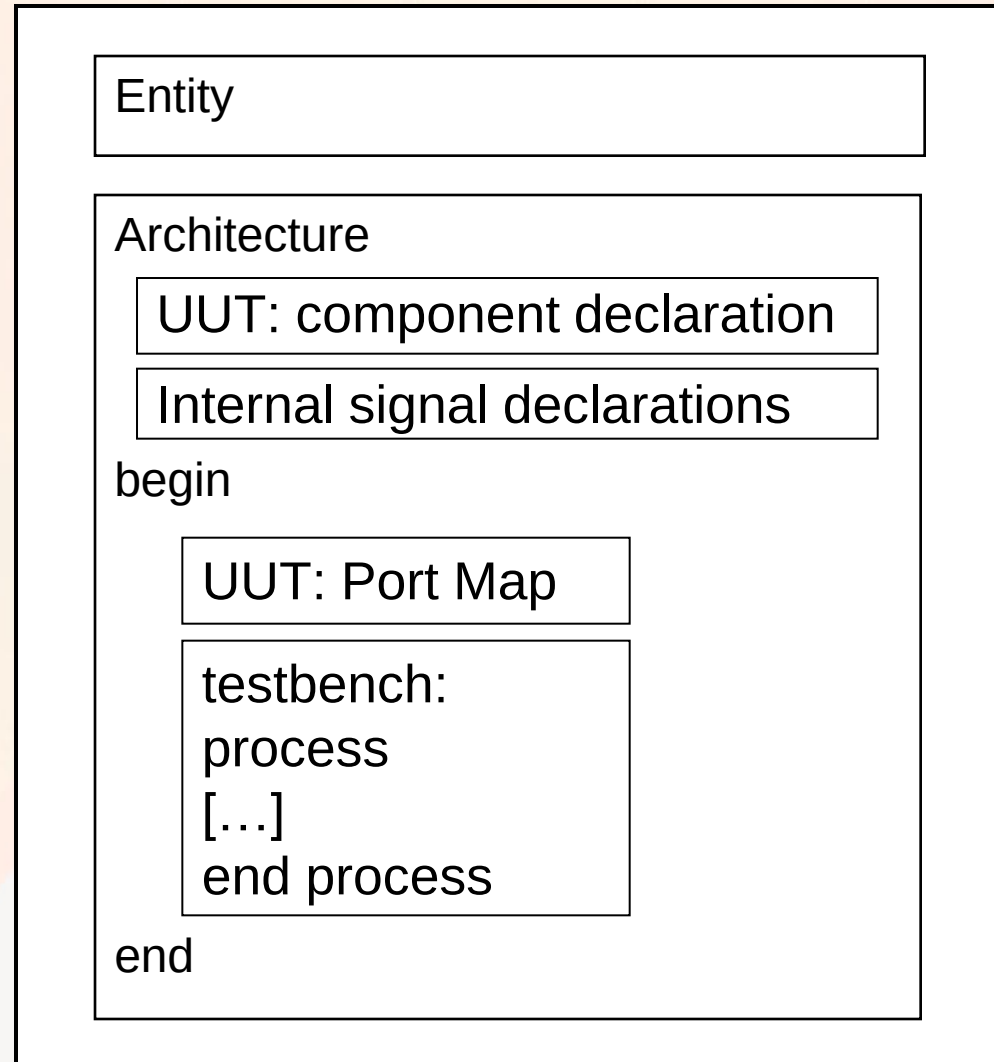
- A *VHDL testbench* is a VHDL module used to verify the logic and timing of a design (the *UUT* or Unit Under Test) by defining *input stimuli* and (optionally) *expected outputs*.



- In other words, they implement the *test strategy* for the design: they are usually applied to all components before they are integrated in a full design (hierarchical design).
- They are often very complex entities!

VHDL testbenches

- A VHDL testbench consists of the Unit Under Test and a series of input stimuli and output checks.
- A VHDL test bench is *not synthesisable* – it is not to be made into a circuit
- The Unit Under Test is included as a *component* in the testbench



VHDL testbenches

- You will be asked to select the VHDL entity to be tested (the UUT).*

```
entity full_adder is
    port (A,B,Cin: in std_logic;
          S, Cout: out std_logic);
end full_adder;

architecture arch of full_adder is
begin
    Cout <= Cout <= ((A xor B) and Cin) or (A and B);
    S <= (A xor B) xor Cin;
end arch;
```

- The tools will generate a "skeleton" testbench:*
 - First, an **entity** with no inputs or outputs*

```
entity test_dut_vhd is
end test_dut_vhd;
```

VHDL testbenches

- A "skeleton" testbench:
 - Then, an *architecture* declaration that includes the *UUT* as a component and *signals* connected to its inputs and outputs

```
architecture behavior of test_dut_vhd is
  -- component declaration for the unit under test (uut)
  component full_adder
    port( a : in  std_logic;
          b : in  std_logic;
          cin : in std_logic;
          s : out std_logic;
          cout : out std_logic);
  end component;
  --inputs
  signal a, b, cin : std_logic := '0';
  --outputs
  signal s, cout : std_logic := '0';
```

Initial value : to be avoided
for synthesizable VHDL!!!!
(but testbenches are NOT synthesizable!)

VHDL testbenches

- A "skeleton" testbench:
 - The UUT component is *instantiated*, the signals are connected and *process* is created.

```
begin
  -- instantiate the unit under test (uut)
  uut: full_adder port map(
    a => a;
    b => b;
    cin => cin;
    s => s;
    cout => cout);

  tb : process
  begin
    -- wait 100 ns for global reset to finish
    wait for 100 ns;
    -- place stimulus here
    wait; -- will wait forever
  end process;
end;
```

The "wait for" statement is NOT SYNTHESIZABLE!!! – but testbenches are not meant to be synthesized.

This delay represents the initial internal FPGA reset – DO NOT REMOVE!!!

VHDL testbenches

- The *stimuli* (inputs) to test your circuit will have to be instantiated within the process in the form of *timed assignments to the inputs of the UUT*

```
tb : process
begin
    -- wait 100 ns for global reset to finish
    wait for 100 ns;
    -- place stimulus here
    a <= '0';
    b <= '0';
    cin <= '0';
    wait for 10ns;
    cin <= '1';
    wait for 10ns;
    cin <= '0';
    b <= '1';
    [...]
    wait; -- will wait forever
end process;
```

Wait for an (almost) arbitrary amount of time (for combinational circuits) – you still want to make it “reasonable”

VHDL testbenches

- For sequential circuits, a separate *clock process* after the UUT declaration is used to generate the clock with an *infinite loop*.

```
[...]
constant clk_period : time := 100ns;
BEGIN
  -- Instantiate the Unit Under Test (UUT)
  uut: my_design PORT MAP( [...], clk=>clk, [...] );

  process -- clock process for CLK
  begin
    CLOCK LOOP : loop
      clk <= '0';
      wait for clk_period/2;
      clk<= '1';
      wait for clk period/2;
    end loop CLOCK_LOOP;
  end process;

  tb : process
    wait for 100ns - reset
    [...]
    wait for clk_period*10;
    [...]
```

Clock period defined
by constant

"loop", "wait for"
are non-synthesizable

Clock signal automatically
extracted from design by wizard

Use of a constant allows clock
frequency to be changed

Delays between input test
patterns should always
defined as multiples (or
fractions) of the clock period

Synthesizable VHDL

- VHDL looks like a programming language
 - Its syntax is derived from Ada, a high-reliability software language
- But it is **NOT** a programming language!!!!
- Writing good VHDL requires a different approach to writing good software
- This is particularly true for **synthesizable** VHDL, where the designer should always consider that everything they write will have to be converted into hardware
- There is an exception to this rule: **testbenches** will never be synthesized, and their code is actually executed (in simulations)

Debugging

- The **ASSERT** statement is used to check output values against expected values.
- If the values are different, then it is possible (optional) to specify a report text that will be printed in the simulation log

```
tb : process
begin
  wait for 100 ns; -- Wait 100 ns for global reset to finish
  RAddress <= "0000";
  for i in 0 to 14 loop
    wait until CLK'event and CLK='1';
    RAddress <= RAddress + 1;
    assert DataOut = "00000000"
    report "RAM not 0 at address: " & integer'image(i)
    severity warning;
  end loop;
end process;
```

This is the expected value

If the actual value is different, print out this message

How severe is the problem?

Note the absence of ";" – this is a single statement!

- **T'image** (*T* is a scalar type) converts to a string
- Severity: [*note*, warning, *error*, *failure*]

Assert for correct operation

- The **ASSERT** statement is designed to "trigger" an action (usually a report) when the actual output differs from the expected. As such, it is aimed at reporting *erroneous behaviour* in the UUT.
- However, it is good design practice in testbenches to complement error messages with *confirmations of correct behaviour*

```
assert DataOut = "00000000"  
report "RAM not 0 at address: " & integer'image(i)  
severity warning;  
assert DataOut /= "00000000"  
report "Contents of RAM checked at address: " & integer'image(i)  
severity note;
```

If the actual value is incorrect,
print out this message

If the actual value is correct,
print out this message

Records

- *A simple way to specify both stimuli and expected responses is to use a table of constants, where each combination of input values and responses is a row in the table.*
- *An efficient way to implement this kind of table is using the **RECORD** type:*

```
type identifier is record
    identifier_list: element_subtype_definition;
    {identifier_list: element_subtype_definition;}
end record;
```

- *To use a record as a table of test vectors, three steps are needed:*
 - *Declare a record type*
 - *Declare an array of records of the record type*
 - *Declare a constant of the array of records type*

Records - example

- To define a set of test vectors for a half adder (no carry in):*

```
type test_vector is record
    a : std_logic;
    b : std_logic;
    sum : std_logic;
    carry_out : std_logic;
end record;

type test_vector_array is array
    (natural range <>) of test_vector;

-- type test_vector_array is array
    (0 to 3) of test_vector;
constant test_vectors : test_vector_array := (
    -- a, b, sum, carry_out
    ('0', '0', '0', '0'),
    ('0', '1', '1', '0'),
    ('1', '0', '1', '0'),
    ('1', '1', '0', '1'));
```

Records - example

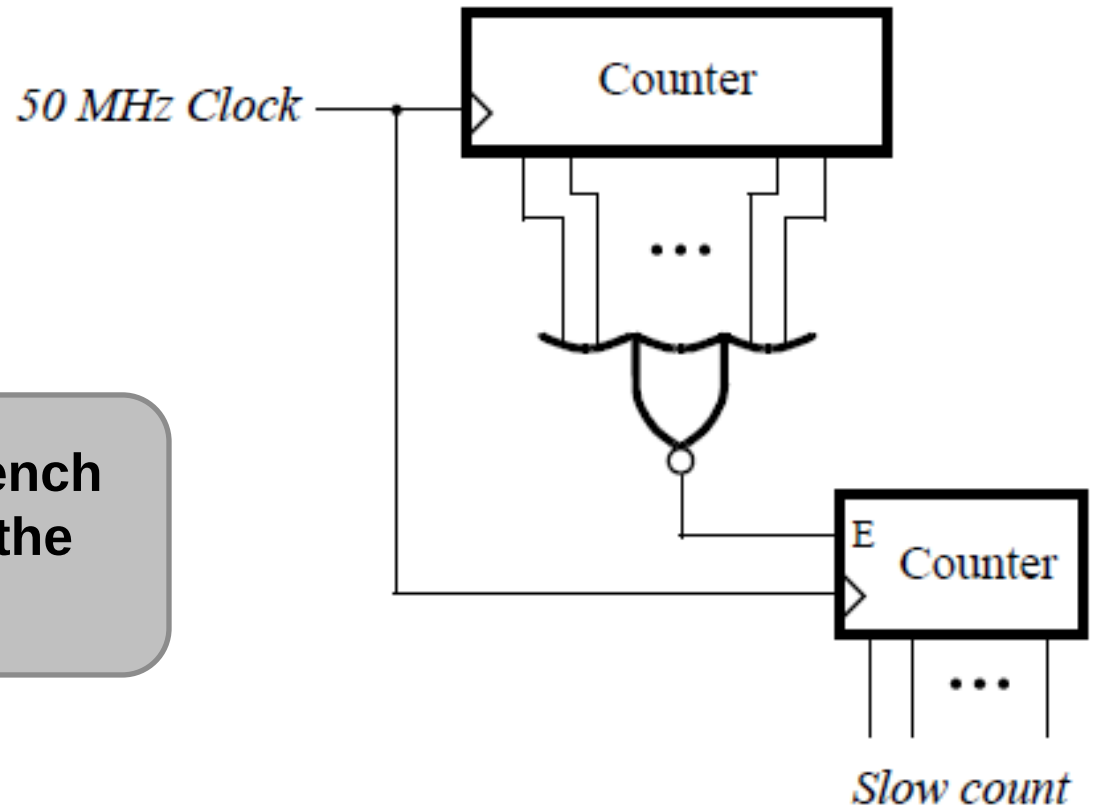
- To use the test vectors within a testbench:*

```
UUT: entity half_adder
    port map (a=>a,b=>b,sum=>sum,carry_out=>carry_out);

verify: process
begin
    for i in test_vectors'range loop
        a <= test_vectors(i).a;
        b <= test_vectors(i).b;
        wait for 20 ns;    -- multiple of period when sequential
        assert ((sum = test_vectors(i).sum) and
                (carry_out = test_vectors(i).carry_out))
        report "Test vector " & integer'image(i) &
            " failed for input a = " & std_logic'image(a) &
            " and b = " & std_logic'image(b)
            severity error;
    end loop;
    wait;
end process;
```

Slow counter

Homework: Write a testbench to verify the operation of the 16-bit register?



Useful links:

- To create skeleton Testbench:
 - https://www.intel.com/content/www/us/en/programmable/quartushelp/13.0/mergedProjects/eda/eda_topics/quartus2/eda_pro_gen_tb.htm
- To Run RTL simulation with TestBench
 - https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/ug/ug_gs_msa_qii.pdf

Thank you!

